Assignment 10

Advection Diffusion

Submitted by

Sanjeev Kumawat
`ph25mscst11026`

Instructor
Anupam Gupta

Date of Submission

April 17, 2026

Contents

1 Introduction **2**

2 Governing Equations **2**

3 Numerical Method **2**

4 Initial and Boundary Conditions **3**

5 Results and Observations **3**

6 Discussion **5**

A Python Code **6**

1 Introduction

Heat transfer is a natural process where heat flows from hotter regions to colder ones, mainly through diffusion. In many real situations, heat is not only spreading but also being carried by a moving fluid, which introduces advection. Understanding the combined effect of diffusion and advection is important for describing real physical systems.
In this work, we study heat transfer in one and two dimensions using numerical methods. First, we analyze pure diffusion to observe how a localized temperature profile smooths out over time. Then, we introduce flow and study how heat is transported and deformed due to advection.
Since exact solutions are difficult, we use finite difference methods for simulation. A key focus is to distinguish between true physical diffusion and additional smoothing introduced by the numerical scheme.

2 Governing Equations

The heat transfer in this study is governed by the diffusion and advection–diffusion equations.
For the one-dimensional case, the temperature evolution is described by the heat equation:

$$\frac{\partial T}{\partial t} = \alpha \frac{\partial^2 T}{\partial x^2}.$$

For two dimensions, the equation extends to:

$$\frac{\partial T}{\partial t} = \alpha \nabla^2 T.$$

When fluid flow is introduced, the governing equation becomes the advection–diffusion equation:

$$\frac{\partial T}{\partial t} + u(y)\frac{\partial T}{\partial x} = \alpha \nabla^2 T.$$

Here, the velocity varies along the y-direction and is given by:

$$u(y) = U_{\max}\left(1 - (2y-1)^2\right).$$

These equations describe how temperature evolves due to diffusion and how it is transported in the presence of flow.

3 Numerical Method

The equations are solved using the finite difference method by discretizing space and time. For diffusion, a forward-time central-space (FTCS) scheme is used, where each point is updated using its neighboring values. In 2D, diffusion is calculated in both directions.
For the advection term, an upwind scheme is used to maintain stability and avoid non-physical oscillations. The time step is chosen carefully to satisfy stability conditions for both diffusion and advection.
This approach provides a stable and efficient way to simulate the evolution of temperature.

4 Initial and Boundary Conditions

In 1D, a localized hot region is placed at the center, with both ends fixed at zero temperature.
In 2D, a circular hot region is initialized at the center, and all boundaries are kept at zero temperature.
For the advection–diffusion case, the same initial condition is used. The inlet is set to zero temperature, the outlet has a zero-gradient condition, and the walls are insulated.
These conditions control how heat spreads, enters, and leaves the system.

5 Results and Observations

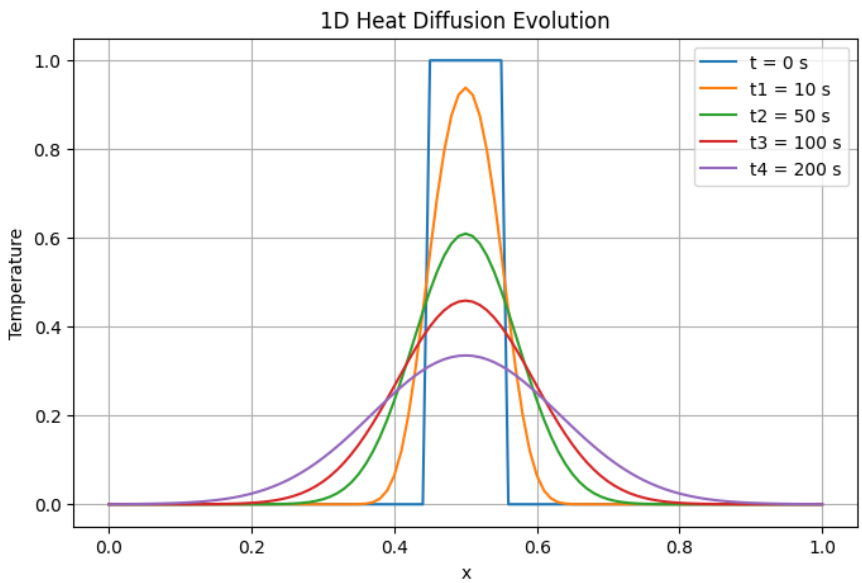

Figure 1

In the 1D case, the initial sharp temperature profile gradually smooths out over time. The peak temperature decreases while the profile spreads outward, showing the effect of diffusion.

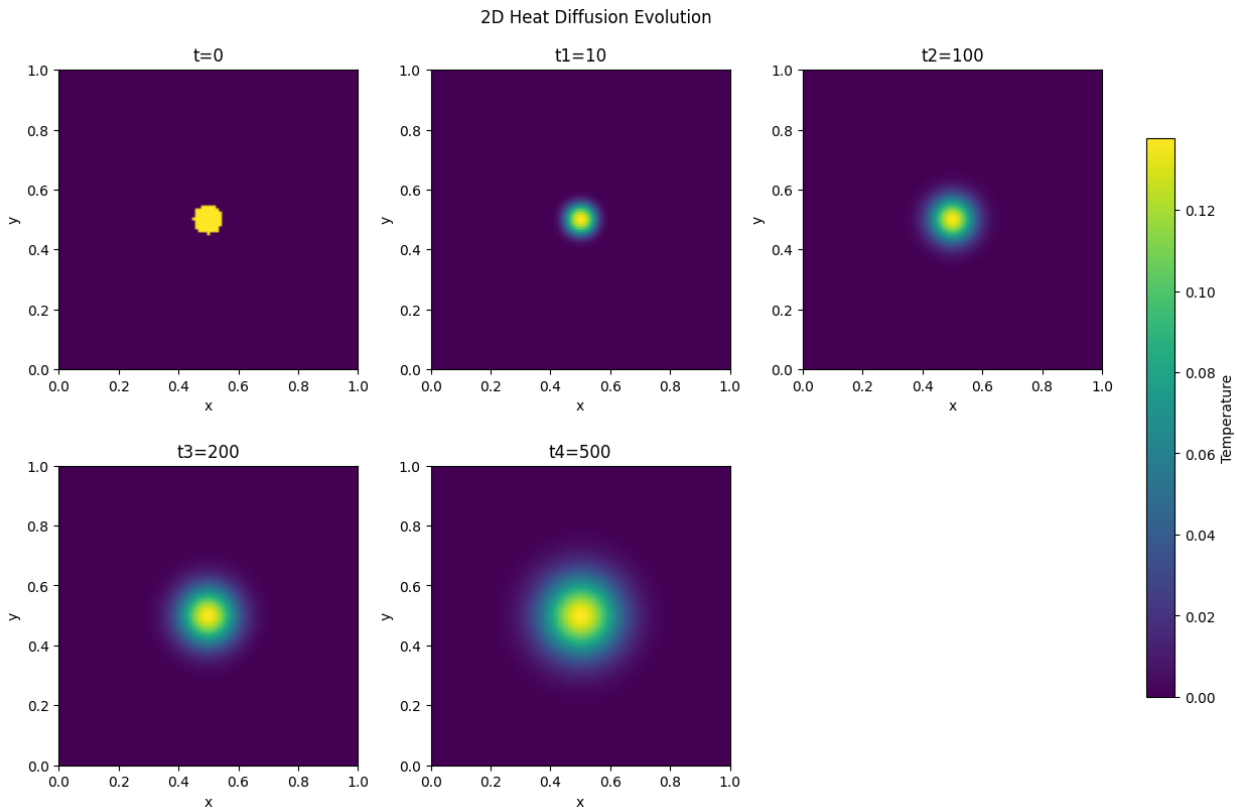

Figure 2

In 2D, heat spreads radially from the central region. The profile becomes wider and smoother, and although it is initially circular, slight distortions appear due to the grid.

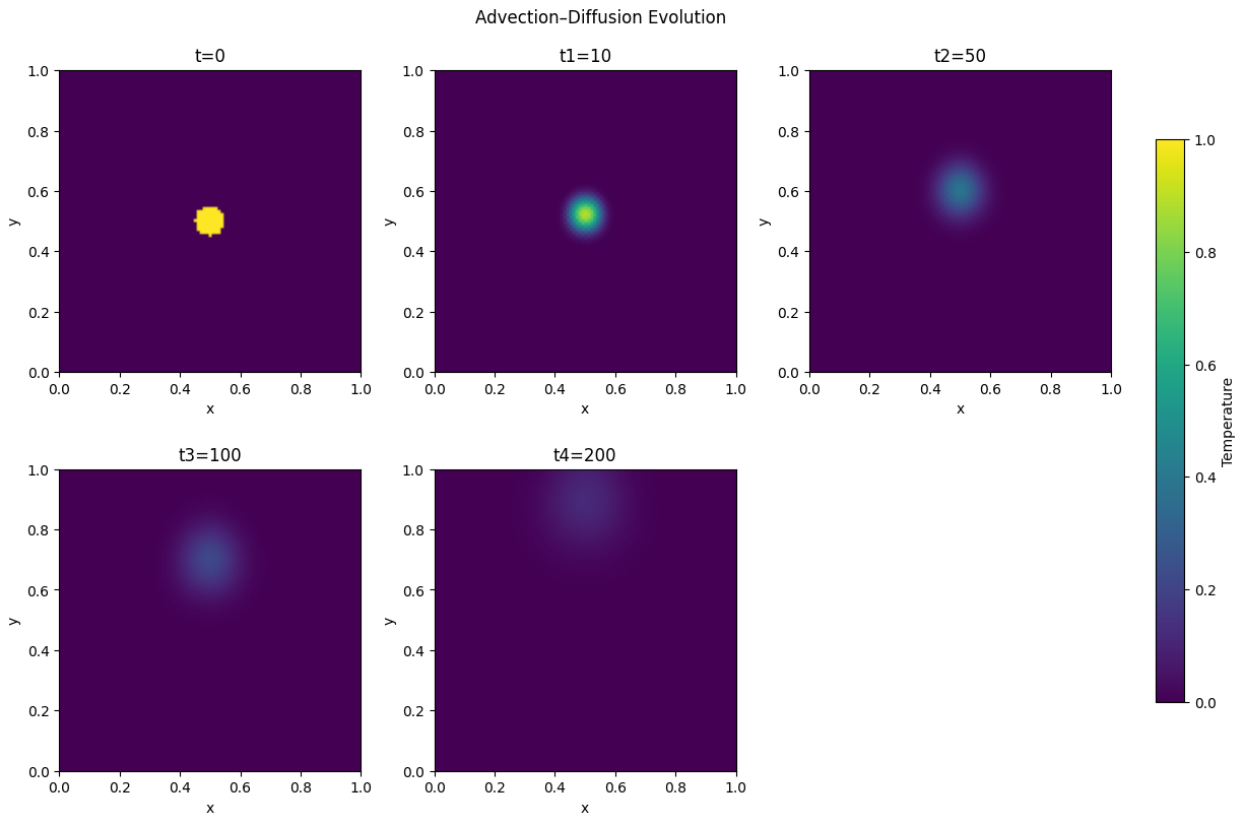

Figure 3

In the advection–diffusion case, the temperature profile is transported in the direction of flow while also spreading. The variation in velocity causes the profile to stretch and deform, while diffusion smooths it over time.
Overall, the results clearly show the combined effects of spreading due to diffusion and transport due to advection.

6 Discussion

The results show that diffusion naturally smooths temperature differences, while advection transports the profile without creating smoothing. In 2D, the spreading is faster due to additional directions of heat flow, and slight distortions appear because of the grid.
The numerical method also introduces extra smoothing, known as numerical diffusion, which can make the profile appear to diffuse faster than expected. This effect becomes more noticeable with larger time steps or coarse grids.
The choice of numerical scheme is important, especially for the advection term. Using an upwind scheme ensures stability but adds numerical diffusion. Therefore, it is important to distinguish between physical behavior and numerical effects while interpreting the results.

A Python Code

```
import numpy as np
import matplotlib.pyplot as plt


#Heat Transfer in 1D
#Parameters
L = 1.0
alpha = 0.01
Nx = 101
dx = L / (Nx - 1)

#Stability condition
dt = 0.4 * dx**2 / alpha  #time step
Nt = 500  #number of time steps

#grid
x = np.linspace(0, L, Nx)

#Initial condition
T = np.zeros(Nx)
T[(x >= 0.45) & (x <= 0.55)] = 1.0

#store for plotting
T_initial = T.copy()
snapshots_1d = [T.copy()]

#Time evolution
for n in range(Nt):
    T_new = T.copy()

    for i in range(1, Nx-1):
        T_new[i] = T[i] + alpha * dt / dx**2 * (T[i+1] - 2*T[i] + T[i-1])

    #Boundary conditions
    T_new[0] = 0
    T_new[-1] = 0

    T = T_new

    #Save few snapshots
    if n in [10, 50, 100, 200]:
        snapshots_1d.append(T.copy())

#plots
plt.figure(figsize=(8,5))
plt.plot(x, T_initial, label="t = 0 s")

times = ["t1 = 10 s", "t2 = 50 s", "t3 = 100 s", "t4 = 200 s"]
for i, snap in enumerate(snapshots_1d[1:]):
    plt.plot(x, snap, label=times[i])

plt.xlabel("x")
```

```
plt.ylabel("Temperature")
plt.title("1D Heat Diffusion Evolution")
plt.legend()
plt.grid()
plt.show()


#2D Heat Diffusion
Nx = Ny = 101
dx = L / (Nx - 1)
dy = dx

#Stability condition (2D)
dt = 0.2 * dx**2 / alpha   # safe choice
Nt = 500

#Grid
x = np.linspace(0, L, Nx)
y = np.linspace(0, L, Ny)

#Temperature field
T = np.zeros((Nx, Ny))

#Initial condition: circular hot region
for i in range(Nx):
    for j in range(Ny):
        if (x[i] - 0.5)**2 + (y[j] - 0.5)**2 < 0.05**2:
            T[i, j] = 1.0

#Store snapshots
snapshots_2d = [T.copy()]

#Time evolution
for n in range(Nt):
    T_new = T.copy()

    for i in range(1, Nx-1):
        for j in range(1, Ny-1):
            T_new[i, j] = T[i, j] + alpha * dt * (
                (T[i+1, j] - 2*T[i, j] + T[i-1, j]) / dx**2 +
                (T[i, j+1] - 2*T[i, j] + T[i, j-1]) / dy**2
            )

    #Boundary conditions (all edges = 0)
    T_new[0, :] = 0
    T_new[-1, :] = 0
    T_new[:, 0] = 0
    T_new[:, -1] = 0

    T = T_new

    #Save snapshots
    if n in [10, 50, 100, 200]:
        snapshots_2d.append(T.copy())
```

```
#Plotting
fig, ax = plt.subplots(2, 3, figsize=(12, 8))
ax = ax.flatten()

times = ["t=0", "t1=10", "t2=50", "t3=100", "t4=200"]

for k in range(5):
    im = ax[k].imshow(snapshots_2d[k], extent=[0,1,0,1], origin='lower')
    ax[k].set_title(times[k])
    ax[k].set_xlabel("x")
    ax[k].set_ylabel("y")

#Hide extra subplot
ax[5].axis('off')

plt.tight_layout()

cbar = fig.colorbar(im, ax=ax[:5], fraction=0.025, pad=0.04)
cbar.set_label("Temperature")

plt.suptitle("2D Heat Diffusion Evolution", y=1.02)
plt.show()


# A d v e c t i o n Diffusion
Umax = 1.0

#Stability conditions
dt_diff = 0.2 * dx**2 / alpha
dt_adv = 0.5 * dx / Umax
dt = min(dt_diff, dt_adv)

Nt = 500

#Velocity profile u(y)
u = Umax * (1 - (2*y - 1)**2)

#Temperature field
T = np.zeros((Nx, Ny))

#Initial condition: circular hot region
for i in range(Nx):
    for j in range(Ny):
        if (x[i] - 0.5)**2 + (y[j] - 0.5)**2 < 0.05**2:
            T[i, j] = 1.0

#Store snapshots
snapshots_adv = [T.copy()]

#Time evolution
for n in range(Nt):
    T_new = T.copy()
```

```python
    for i in range(1, Nx-1):
        for j in range(1, Ny-1):

            #Advection term (Upwind scheme)
            adv = u[j] * (T[i, j] - T[i-1, j]) / dx

            #Diffusion term
            diff = alpha * (
                (T[i+1, j] - 2*T[i, j] + T[i-1, j]) / dx**2 +
                (T[i, j+1] - 2*T[i, j] + T[i, j-1]) / dy**2
            )

            #Update equation
            T_new[i, j] = T[i, j] - dt * adv + dt * diff

    #Boundary conditions

    #Inlet (x = 0)
    T_new[0, :] = 0

    #Outlet (x = L)     zero gradient
    T_new[-1, :] = T_new[-2, :]

    #Walls (y = 0, 1)     insulated
    T_new[:, 0] = T_new[:, 1]
    T_new[:, -1] = T_new[:, -2]

    T = T_new

    #Save snapshots
    if n in [10, 50, 100, 200]:
        snapshots_adv.append(T.copy())

#Plotting
fig, ax = plt.subplots(2, 3, figsize=(12, 8))
ax = ax.flatten()

times = ["t=0", "t1=10", "t2=50", "t3=100", "t4=200"]

for k in range(5):
    im = ax[k].imshow(
        snapshots_adv[k],
        extent=[0,1,0,1],
        origin='lower',
        cmap='viridis',
        vmin=0,
        vmax=1
    )
    ax[k].set_title(times[k])
    ax[k].set_xlabel("x")
    ax[k].set_ylabel("y")

ax[5].axis('off')
```

```
plt.tight_layout()

cbar = fig.colorbar(im, ax=ax[:5], fraction=0.025, pad=0.04)
cbar.set_label("Temperature")

plt.suptitle("Advection Diffusion Evolution", y=1.02)
plt.show()
```

References

[1] Sanjeev K, "Python code used in this report," GitHub repository, 2026. Available at: `https://github.com/arsanjeevk/Computational_Methods/tree/main/Assignment-11(Advection_Diffusion)` [Accessed: 17 April 2026].